

TP Éléments finis

I. EXERCICES PRÉLIMINAIRES:

A. Création de matrices particulières

1. Écrire en Python la fonction qui crée, en fonction de n , la matrice pentadiagonale A telle que:

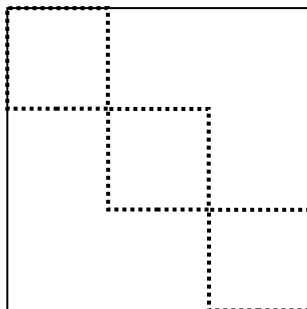
$$A_{ij} = \begin{cases} 4 & \text{si } i=j \\ -1 & \text{si } i=j+1 \\ -1 & \text{si } i=j-1 \\ 2 & \text{si } i=j+n \\ 2 & \text{si } i=j-n \end{cases} \quad \text{avec } i=1..n^2, j=1..n^2.$$

On prendra $n = 3$. (On utilisera la fonction **diag** de la bibliothèque numpy.)

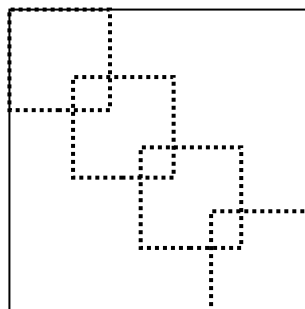
2. Créer une matrice (9x9) diagonale par blocs en sommant des sous-blocs A de taille (3x3) avec

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} \text{ telle que:}$$

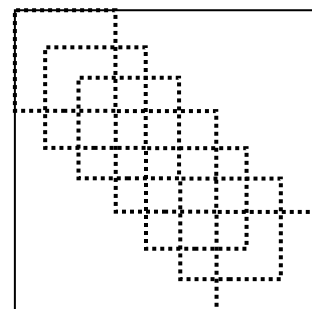
- a- les sous-blocs sommés soient décalés de 3 lignes et de 3 colonnes à chaque itération,
- b- les sous-blocs sommés soient décalés de 2 lignes et de 2 colonnes à chaque itération,
- c- les sous-blocs sommés soient décalés d'une ligne et d'une colonne à chaque itération.



(a)



(b)



(c)

B. Tracé de graphiques

Tracer un nuage de points obtenus aléatoirement (fonction **random** de la bibliothèque random), en échelle linéaire puis en échelle logarithmique. Bien spécifier les légendes, titres, etc ...

C. Programmation de la méthode inversion de système linéaire par factorisation LU

Soit un système linéaire $AX=B$ avec A une matrice $(n \times n)$, B un vecteur colonne de dimension n et X le vecteur solution recherché. La méthode de factorisation LU permet de réécrire le système linéaire sous la forme $LUX = B$. On a donc $A=LU$ avec L une matrice triangulaire inférieure de diagonale unitaire et U une matrice triangulaire supérieure. La résolution du système $AX=B$ revient alors à résoudre deux systèmes triangulaires :

$$AX=B \Rightarrow LUX=B \Rightarrow \begin{cases} LY=B \\ UX=Y \end{cases},$$

ce qui est moins coûteux en temps de calcul, en particulier lorsque l'on doit résoudre plusieurs systèmes linéaires avec différents seconds membres, mais la même matrice A .

Traduire l'algorithme de factorisation suivant en une fonction python dont l'entrée est la matrice A et les sorties les matrices L et U :

Initialisation:

$$\begin{aligned} U_{ij} &= 0 && \text{pour } i = 0, n-1 \text{ et } j = 0, n-1 \text{ avec } n = \text{taille}(A) \\ L_{ij} &= \delta_{ij} && \text{pour } i = 0, n-1 \text{ et } j = 0, n-1 \text{ avec } n = \text{taille}(A) \end{aligned}$$

Factorisation:

$$\begin{aligned} &\text{Pour } j=0, n-1 \\ &\quad \text{Pour } i=0, j \\ &\quad \quad U_{ij} = A_{ij} - \sum_{k=0}^{i-1} L_{ik} U_{kj} \\ &\quad \text{Fin} \\ &\quad \text{Pour } i=j+1, n-1 \\ &\quad \quad L_{ij} = \frac{1}{U_{jj}} \left(A_{ij} - \sum_{k=0}^{j-1} L_{ik} U_{kj} \right) \\ &\quad \text{Fin} \\ &\text{Fin} \end{aligned}$$

Pour la résolution, on utilise donc successivement deux algorithmes : un algorithme dit de **descente** pour lequel on calcule le vecteur Y tel que $LY=B$ et un algorithme dit de **remontée** pour lequel on calcule le vecteur X tel que $UX=Y$.

Écrire les deux fonctions Python correspondant à ces algorithmes.

Résolution:

- Descente $LY=B$:

$$\begin{aligned} &\text{Pour } i = 0, n-1 \\ &\quad Y_i = B_i - \sum_{k=0}^{i-1} L_{ik} Y_k \\ &\quad \text{Fin} \end{aligned}$$

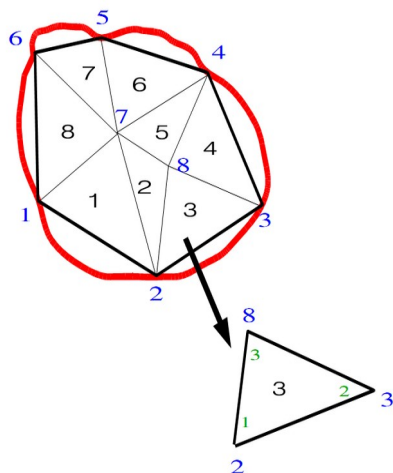
- Remontée $UX=Y$:

$$\begin{aligned} &\text{Pour } i = n-1, 0 \\ &\quad X_i = \frac{1}{U_{ii}} \left(Y_i - \sum_{k=i+1}^{n-1} U_{ik} X_k \right) \\ &\quad \text{Fin} \end{aligned}$$

D. Principe de la structure des données pour les éléments finis:

1. La table de connexion:

Dans un maillage éléments finis, les éléments sont numérotés de 1 à N_e et les nœuds de 1 à N_n . Chaque nœud est défini par ses coordonnées globales répertoriées dans la table de coordonnées globales des nœuds géométriques. Chaque élément est défini par la liste de ses nœuds géométriques relevés dans le sens trigonométrique. Cette liste est stockée dans la table de connexion.



N° Noeud	x	y
1	0	0
2	2	-1.5
3	4	0
4	3	3
5	1	3.5
6	0	3
7	1.5	1.5
8	2.5	1

Table de coordonnées globales des noeuds

N° Triangle	s.l* n°1	s.l n°2	s.l n°3
1	1	2	7
2	2	8	7
3	2	3	8
4	3	4	8
5	4	7	8
6	4	5	7
7	5	6	7
8	1	7	6

* : sommet local

Table de connexion

En utilisant la fonction **tables** fournie dans le fichier table.py, créez une table de connexion ainsi qu'une table de coordonnées pour un maillage structuré avec des nœuds disposés en un quadrillage de 3 nœuds suivant x ($N_x = 3$) et 5 nœuds suivant y ($N_y = 5$), observez leur construction, leur taille et leur contenu.

2. Tracé de maillages

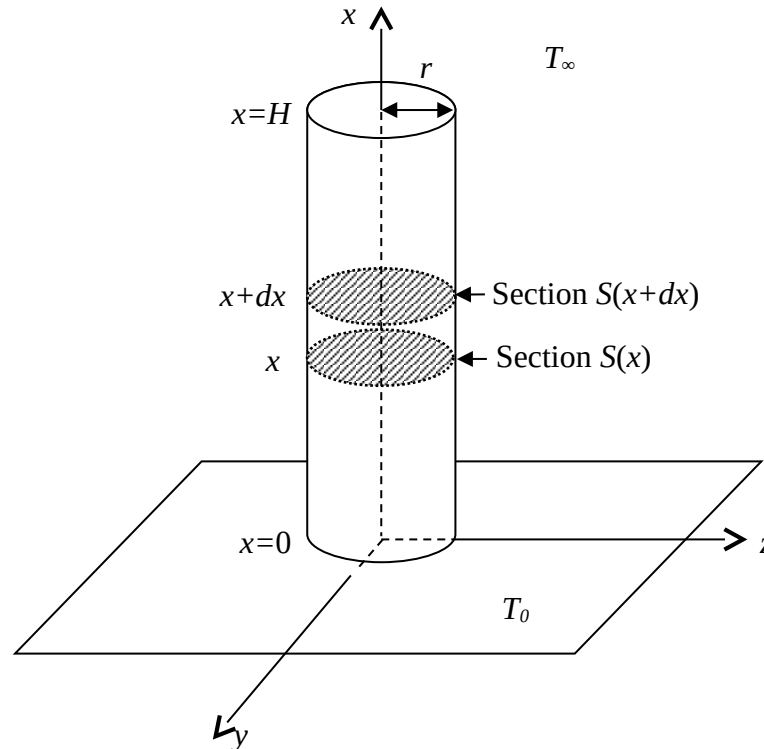
Faire une fonction Python **trace** qui trace un triangle à partir des coordonnées des trois sommets.

Faire une fonction **maillage** qui trace un maillage avec la fonction précédente et la table de connexion et de coordonnées précédemment calculées.

II. PROBLÈME DE CONVECTION / CONDUCTION EN RÉGIME STATIONNAIRE

On cherche à obtenir la répartition de la température dans un barreau métallique (type ailette de refroidissement d'un processeur). Le barreau a une longueur $H = 5 \text{ mm}$ et le rayon de la section est $r = 0.5 \text{ mm}$.

La géométrie du problème est présentée sur la figure suivante :



La température sur l'extrémité du barreau en $x = 0$ est imposée et égale à $T_0 = 100^\circ\text{C}$. A l'autre extrémité, on a placé un matériau isolant (paroi adiabatique, flux thermique nul).

L'ailette est refroidie par convection avec le milieu ambiant ($h = 100 \text{ W.m}^{-2}.\text{°C}^{-1}$, $T_\infty = 20^\circ\text{C}$).

Donnée : Conductivité thermique $\lambda = 40 \text{ W.m}^{-1}.\text{°C}^{-1}$.

En écrivant que la somme des flux thermiques sur un petit élément du barreau d'épaisseur dx doit être nulle, déterminer l'équation d'équilibre du système et ses conditions aux limites.

ÉLÉMENTS FINIS UNIDIMENSIONNELS LINÉAIRES

On considère dans un premier temps que le problème est 1D.

1. Écrire le résidu et la formulation faible de ce problème.
2. Déterminer l'expression des fonctions de base pour des éléments finis linéaires aussi appelés P^1 . En déduire la formulation faible discrète.
3. On fixe le nombre d'éléments $Ne = 5$. La longueur de l'intervalle de discrétisation est $dx = x_{i+1} - x_i$. Calculer les matrices de raideur et de masse élémentaires.
4. Assembler la matrice globale à partir des matrices élémentaires.
5. Appliquer les conditions aux limites sur la matrice assemblée et le second membre.
6. Écrire le programme Python pour résoudre ce problème. Tracer le profil de température.

7. La solution exacte est obtenue au moyen du calcul formel :

$$T_{ex}(x) = T_{\infty} + (T_0 - T_{\infty}) \frac{\cosh(m(H-x))}{\cosh(mH)} \text{ avec } m = \sqrt{\frac{hP}{\lambda S}}$$

et P le périmètre d'une section du barreau.

Étudier l'ordre de précision de la méthode des éléments finis P1 en traçant la courbe de l'erreur relative ϵ en fonction de dx (longueur d'un élément) dans un graphe en échelle logarithmique.

On prendra
$$\epsilon = \frac{\|U_{num}(x) - U_{ex}(x)\|}{\|U_{ex}(x)\|} = \frac{\sqrt{\sum_{i=0}^{Ne} (U_{num}(x_i) - U_{ex}(x_i))^2}}{\sqrt{\sum_{i=0}^{Ne} U_{ex}(x_i)^2}}$$

(voir la fonction **norm** de la bibliothèque numpy.linalg).

On retire désormais l'isolant thermique placé sur l'extrémité en $x = H$. La condition aux limites n'est donc plus adiabatique. Il y a échange avec le milieu extérieur par convection avec un coefficient d'échange $h' \neq h$. On a $h' = 80 \text{ W.m}^{-2}.\text{°C}^{-1}$.

8. Exprimer la nouvelle condition aux limites et les modifications induites sur le système d'équations.

9. Écrire le programme Python pour résoudre ce problème. Tracer le profil de température.

10. La solution exacte est:

$$T_{ex}(x) = T_{\infty} + (T_0 - T_{\infty}) \frac{\cosh(m(H-x)) + \frac{h'}{\lambda m} \sinh(m(H-x))}{\cosh(mH) + \frac{h'}{\lambda m} \sinh(mH)} \text{ avec } m = \sqrt{\frac{hP}{\lambda S}}.$$

Tracer la courbe de l'erreur relative ϵ en fonction de dx dans un graphe en échelle logarithmique et en déduire l'ordre de précision.

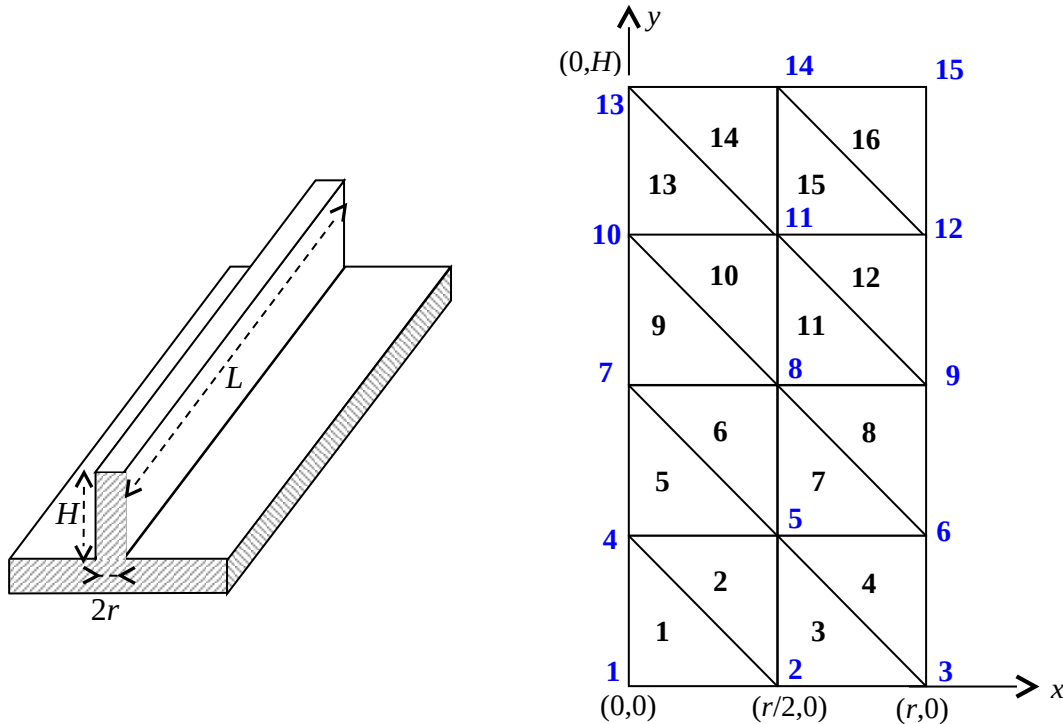
ÉLÉMENTS FINIS UNIDIMENSIONNELS QUADRATIQUES

Reprendre le travail précédent en utilisant des éléments finis quadratiques. On pourra utiliser MAPLE ou la bibliothèque sympy de calcul formel en Python pour calculer plus rapidement les composantes des matrices élémentaires. Quel est l'ordre de précision de la méthode des éléments finis P²?

ÉLÉMENTS FINIS BIDIMENSIONNELS LINÉAIRES:

A. Maillage triangulaire, éléments finis P^1

On étudie maintenant le profil de température 2D dans une coupe d'une ailette de grande longueur L par rapport à sa hauteur H et largeur $2r$. Pour des raisons de symétrie, on limitera le domaine d'étude à $[0,r] \times [0,H]$. On utilise le maillage triangulaire suivant :



Récupérer les tables de coordonnées et de connexion associées avec la fonction **tables**.

1. Écrire l'équation d'équilibre du système, ses conditions aux limites et la formulation faible de ce problème 2D.
2. On utilise la méthode des éléments finis linéaires P^1 . En utilisant les résultats du TD, écrire la formulation faible discrète ainsi que les matrices de raideur élémentaires.

Principe général d'assemblage: algorithme d'assemblage avec la table de connexion :

Pour l'assemblage des matrices élémentaires Ke dans une matrice globale K_{global} en utilisant une table de connexion *table_connexion* pour un problème éléments finis en 2D avec des éléments triangulaires, on utilise l'algorithme suivant:

```

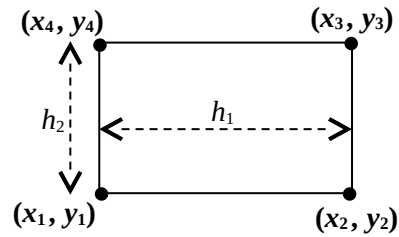
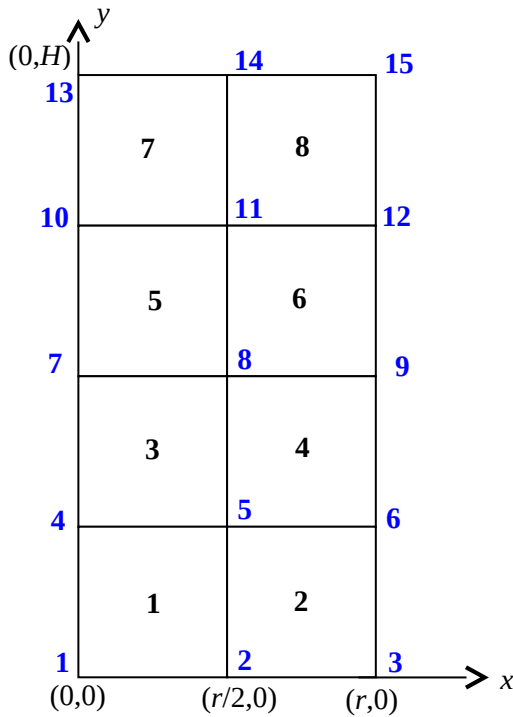
Pour k=1, Ne
    Calcul de Ke
    Pour i=1,3
        Pour j=1,3
            Ig = table_connexion(k,i)
            Jg = table_connexion(k,j)
            Kglobal (Ig,Jg) = Kglobal (Ig,Jg) + Ke(i,j)
        Fin Pour
    Fin Pour
Fin Pour
    
```

3. Écrire le programme Python pour résoudre ce problème. Tracer le profil de température 2D. Pour le tracé du profil de température, on peut utiliser la fonction **contour** ou **contourf** de la bibliothèque matplotlib.pyplot.
4. Généraliser le programme de manière à ce qu'il puisse fonctionner avec n'importe quel maillage.

B. Maillage quadrangulaire, éléments finis Q^1

La seconde façon de mailler le domaine est d'utiliser des quadrangles. Les maillages quadrangulaires sont beaucoup utilisés, surtout en mécanique des solides car ils peuvent être plus précis. Cependant ils proposent des possibilités de maillage moins générales que les triangles.

1. En suivant la même méthode que pour le maillage triangulaire, écrire les tables de coordonnées et de connexion associées au maillage quadrangulaire ci-dessous. On considère ici que tous les quadrangles sont des rectangles de même hauteur h_2 et de même largeur h_1 .



$$\Omega = Q_1 \cup Q_2 \cup \dots \cup Q_{Nk}$$

avec Nk le nombre de quadrangles

Sur ce maillage, l'approximation est donnée par une somme de fonctions bilinéaires en x et y :

$$\tilde{\theta}(x, y) = \sum_{k=1}^{Nk} \tilde{\theta}^k(x, y)$$

$$\text{avec } \tilde{\theta}^k(x, y) = \begin{cases} \theta_{n_1} H_1(x, y) + \theta_{n_2} H_2(x, y) + \theta_{n_3} H_3(x, y) + \theta_{n_4} H_4(x, y) & \text{si } (x, y) \in Q_k \\ 0 & \text{sinon} \end{cases}$$

Les fonctions de forme sont de la forme:

$$H_i(x, y) = a_i + b_i x + c_i y + d_i xy \text{ pour } i=1,4$$

et vérifient les propriétés suivantes:

$$H_i(x_{n_j}, y_{n_j}) = \begin{cases} 1 & \text{si } i=j \\ 0 & \text{sinon} \end{cases}$$

Dans ces conditions et en suivant le même développement que dans le cas d'un maillage triangulaire, on obtient les matrices de raideur suivantes dans le cas particulier d'un maillage parallèle avec les axes du plan :

$$K^x = \frac{h_2}{6h_1} \begin{bmatrix} 2 & -2 & -1 & 1 \\ -2 & 2 & 1 & -1 \\ -1 & 1 & 2 & -2 \\ 1 & -1 & -2 & 2 \end{bmatrix} \quad K^y = \frac{h_1}{6h_2} \begin{bmatrix} 2 & 1 & -1 & -2 \\ 1 & 2 & -2 & -1 \\ -1 & -2 & 2 & 1 \\ -2 & -1 & 1 & 2 \end{bmatrix} \quad \text{et} \quad K = K^x + K^y$$

2. Modifier le programme Python précédent pour résoudre le problème avec des éléments finis Q^1 . Tracer le profil de température 2D.
3. Généraliser ce code pour pouvoir faire évoluer le nombre de points de maillage suivant x et y et en déduire la valeur de la température obtenue en $(0, H)$. Tracer $\tilde{T}(0, h)$ en fonction de h_I en gardant constant le rapport h_I/h_2 . Que peut-on conclure?

III. CAS D'UNE EDO À COEFFICIENTS VARIABLES

On cherche à résoudre un problème aux conditions aux limites 1D par la méthode des éléments finis. Le problème s'écrit sous la forme de l'équation différentielle ordinaire suivante:

$$x^2 \frac{d^2 u}{dx^2} + 2x \frac{du}{dx} + 4 = 0 \quad \text{pour} \quad 1 < x < 4$$

associée aux conditions aux limites suivantes: $u(1) = 1$ et $u(4) = 0$.

1. Écrire le résidu et la formulation faible de ce problème.
2. Déterminer l'expression des fonctions de forme pour des éléments finis linéaires.
3. On fixe le nombre d'éléments $N_e = 5$. La longueur de l'intervalle de discrétisation est $h = x_{i+1} - x_i$. Calculer les matrices de raideur et de masse élémentaires pour chaque élément.
4. Assembler la matrice globale à partir des matrices élémentaires.
5. Appliquer les conditions aux limites sur la matrice assemblée et le second membre.
6. Écrire le programme Python pour résoudre ce problème.
7. La solution exacte est obtenue au moyen du calcul formel :

$$u_{ex}(x) = -4 \ln(x) - \frac{\frac{32}{3} \ln(2) - \frac{4}{3}}{x} - \frac{1}{3} + \frac{32}{3} \ln(2).$$

Étudier l'ordre de précision de la méthode des éléments finis P^1 en traçant la courbe de l'erreur relative ϵ en fonction de dx dans un graphe en échelles logarithmiques.